

Assignment 1

2024117009

Cpu:

```

0s 0 lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 42 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 24
On-line CPU(s) list: 0-23
Vendor ID: GenuineIntel
Model name: Intel(R) Core(TM) Ultra 9 275HX
CPU family: 6
Model: 198
Thread(s) per core: 1
Core(s) per socket: 24
Socket(s): 1
Stepping: 2
Microcode version: 0x122
CPU(s) scaling MHz: 29%
CPU max MHz: 5400.0000
CPU min MHz: 800.0000
BogoMIPS: 6144.00
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
      pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx
      pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_goo
      d nopl xtopology nonstop_tsc cpuid aperfmperf tsc_known_freq pni
      pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 sse3 sdbg fma cx
      16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_
      timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_f
      ault ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow flexpriority e
      pt vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpc
      id rdt_a rdseed adx smap clflushopt clwb intel_pt sha_ni xsaveopt
      xsavec xgetbv1 xsaves split_lock_detect user_shstk avx_vnni lass
      lam wbnoinvd dtherm ida arat pln pts hwp hwp_notify hwp_act_wind
      ow hwp_epp hwp_pkg_req hfi vnmi umip pku ospke waitpkg gfnl vaes
      vpclmulqdq rdpid bus_lock_detect movdiri movdir64b fsrm md_clear
      serialize arch_lbr ibt flush_l1d arch_capabilities

Virtualization features:
  Virtualization: VT-x

Caches (sum of all):
L1d: 768 KiB (20 instances)
L1i: 1.3 MiB (20 instances)
L2: 40 MiB (12 instances)
L3: 36 MiB (1 instance)

NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-23

```

RAM:

```

└─ 0s ○ free -h
total          used          free
Mem:          30Gi          9.16i          8.26i
Swap:         30Gi          1.7Mi          30Gi

└─ 0s ○
└─ DIP_Assignment_1 ▲ | ●○○ | py 1L 3.14.7 20:29
shared buff/cache available
1.96i          146i          216i
└─ DIP_Assignment_1 ▲ | ●○○ | py 1L 3.14.7 20:30

```

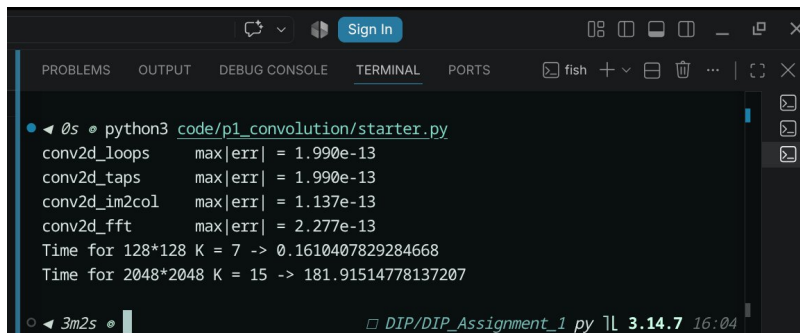
Numpy Version:

```

↳ 0s @ python3 -c "import numpy; print(numpy.__version__)"
2.5.2
↳ 0s @ 

```

1.1

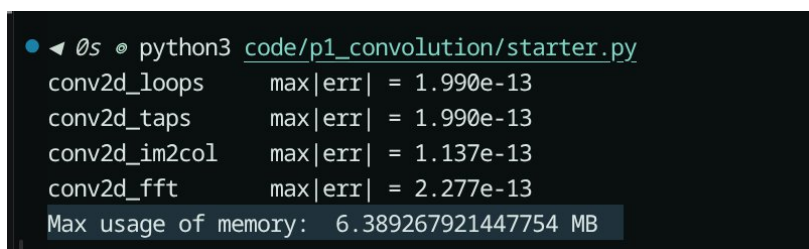


```
0s • python3 code/p1_convolution/starter.py
conv2d_loops      max|err| = 1.990e-13
conv2d_taps       max|err| = 1.990e-13
conv2d_im2col     max|err| = 1.137e-13
conv2d_fft        max|err| = 2.277e-13
Time for 128*128 K = 7 -> 0.1610407829284668
Time for 2048*2048 K = 15 -> 181.91514778137207
3m2s • DIP/DIP_Assignment_1.py 3.14.7 16:04
```

We were asked to run the loops method only for small N and K values like 128*128 and 7*7 as the loops method is $O(n*m*kx*ky)$ where $n*m$ is the dimension of the image and kx ky are the dimension of the image. As it doesn't use any of the numpy vectorization optimization it is very slow scales poorly.

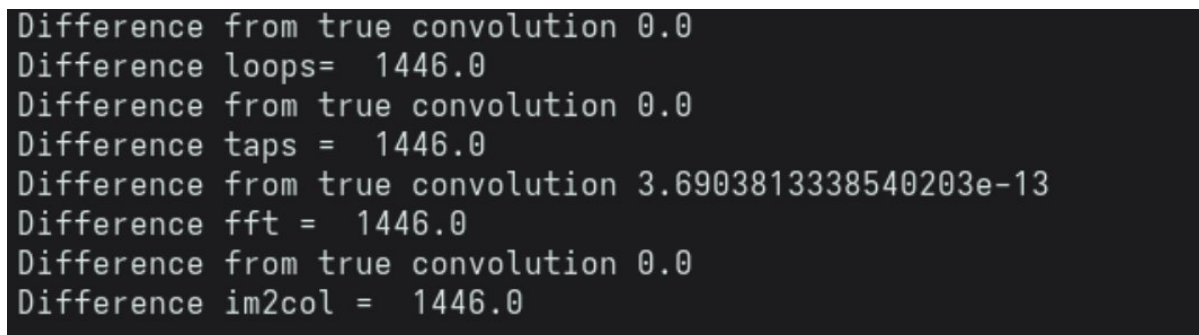
Extrapolation for 2048*2048, K =1 5:

so the time taken for the 2048*2048 with K = 15 should be around 1175.51.times the time for 128*128*7*7 = 16*16*2*2 (15~= 2^3) which is 1024. $0.161040 * 1175.51 = 189.3$ seconds which is similar to the test result of 181 seconds



```
0s • python3 code/p1_convolution/starter.py
conv2d_loops      max|err| = 1.990e-13
conv2d_taps       max|err| = 1.990e-13
conv2d_im2col     max|err| = 1.137e-13
conv2d_fft        max|err| = 2.277e-13
Max usage of memory: 6.389267921447754 MB
```

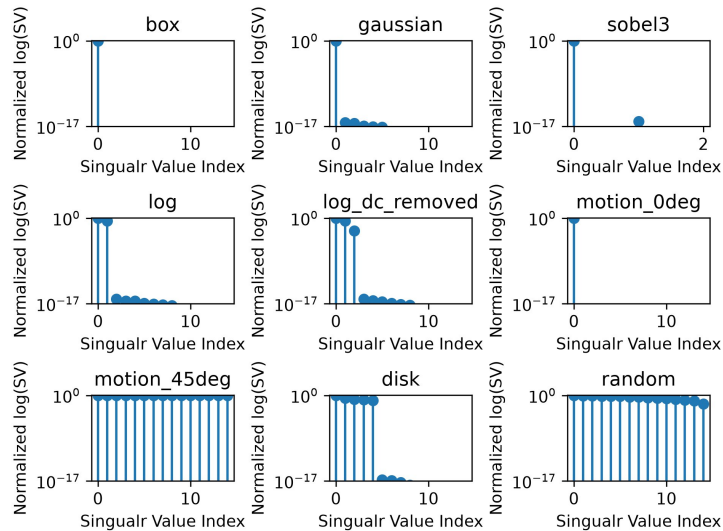
The max usage of memory for im2col is 6MB. The sliding window shape which is part of the im2col function creates a np array of shape $(128 \times 128, 7 \times 7) = (16384, 49)$. This is the major space taking part of the program which takes $16384 \times 49 \times 8$ bytes ≈ 6 mb



```
Difference from true convolution 0.0
Difference loops= 1446.0
Difference from true convolution 0.0
Difference taps = 1446.0
Difference from true convolution 3.6903813338540203e-13
Difference fft = 1446.0
Difference from true convolution 0.0
Difference im2col = 1446.0
```

We can see how all 4 implemented functions show substantial difference $\max|\text{convolution-correlation}| = 1460$, when comparing correlation to convolution for an asymmetric kernel.

1.2



Analysis:

In class we did that convolution is linear. To solve we have done a single value decomposition of the image. SVD breaks down the 2D kernel into several rank 1 separable ranked by the importance. The singular values in some sense shows the importance of each rank-1 component. Now the tol (threshold) if we choose a very small value of threshold then the floating error point error (cause by inaccuracy of representation of decimals in python) can be counted as an important rank-1 component (will increase rank). If we chose too high of a value for this threshold then some of the important components might not be counted (will decrease rank). https://numpy.org/doc/stable/reference/generated/numpy.linalg.matrix_rank.html

In the official documentation of the numpy rank it is written the default value of the tol is $S_{\max} \cdot \max(M, N) \cdot \text{eps}$

now as the problem is for 15×15 M, N = 15 and the eps value is the machine epsilon for python float64 this gives us a value of $\sim 9 \times 10^{-15}$ hence we used 10^{-14} tolerance value

Rank:

Box = 1

Gaussian = 1

Sobel3 = 1

LoG = 2

DC_removed_Log = 3

motion_0 = 1

motion_45 = 15

disk=5

random=15

Graph analysis:

box: has rank 1 and the graph shows how the SV value for index 0 is 1 and the rest are negligible 10^{-17}

Gaussian: has rank 1 and similar to box graph shows SV value for index 0 = 1 and the rest are negligible

Sobel3 has a size of 3×3 and hence only goes till index 2, index0 has value = 1 and rest are

negligible

log has a rank of 2. The graph reflects this as index 0 and index 1 have significant values and rest are negligible

log_dc_removed has rank 3: the graph reflects this as index 0, 1, 2 have significant values and rest are negligible

Full rank matrices like motion45deg have rank = 15 as it is a scaled identity matrix and this is also visible in the graph where we see values for all 15 indices being significant.

Finally the disk has a rank of 5 which shows it is not separable but can be approximated

Laplacian of Gaussian

$$g(x) = e^{-\frac{x^2}{2\sigma^2}}$$

$$g'(x) = -\frac{1}{2\sigma^2} \cdot 2x \cdot e^{-\frac{x^2}{2\sigma^2}} \rightarrow -\frac{x}{\sigma^2} e^{-\frac{x^2}{2\sigma^2}}$$

$$g''(x) = -\frac{x}{\sigma^2} \cdot \frac{-x}{\sigma^2} e^{-\frac{x^2}{2\sigma^2}} + e^{-\frac{x^2}{2\sigma^2}} \cdot \frac{-1}{\sigma^2}$$

$$= \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2} \right) e^{-\frac{x^2}{2\sigma^2}}$$

now in 2D

$$\left(\frac{x^2+y^2}{\sigma^4} - \frac{1}{\sigma^2} \right) e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

$$= \frac{1}{\sigma^4} \left[\frac{x^2}{\sigma^2} - 1 \right] e^{-\frac{x^2}{2\sigma^2}} \cdot e^{-\frac{y^2}{2\sigma^2}} + \frac{1}{\sigma^4} \left[\frac{y^2}{\sigma^2} - 1 \right] e^{-\frac{y^2}{2\sigma^2}} \cdot e^{-\frac{x^2}{2\sigma^2}}$$

for vrow (fundct)

$$\Rightarrow C_1 \cdot e^{-\frac{y^2}{2\sigma^2}} + C_2 y^2 \cdot e^{-\frac{y^2}{2\sigma^2}}$$

which are 2 independent vectors
hence rank = 2

Actually easier proof

$$\nabla^2 g = \frac{d^2 g}{dx^2} + \frac{d^2 g}{dy^2} \text{ as 2 separable terms rank 2}$$

(a)

(b) $\text{rank}(A) + \text{rank}(B) \geq \text{rank}(A+B)$

rank LOG has rank 2. Subtracting mean is equivalent to adding a constant matrix with $A_{i,j} = -\mu$

$$\therefore A_{n \times n} = -\mu \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} \begin{bmatrix} 1, 1, 1, \dots, 1 \end{bmatrix}_n$$

$\therefore$ The constant matrix has rank 1

Upper bound

$$\text{rank}(\text{LOGoc}) \leq 2+1=3$$

Now for rank to be 2 or lower the column space of the constant (the vector of all 1s) is contained in column space of LOG.

$$\left(\frac{x^2}{\sigma^2} - 1 \right) e^{-\frac{x^2}{2\sigma^2}} \quad e^{-\frac{x^2}{2\sigma^2}}$$

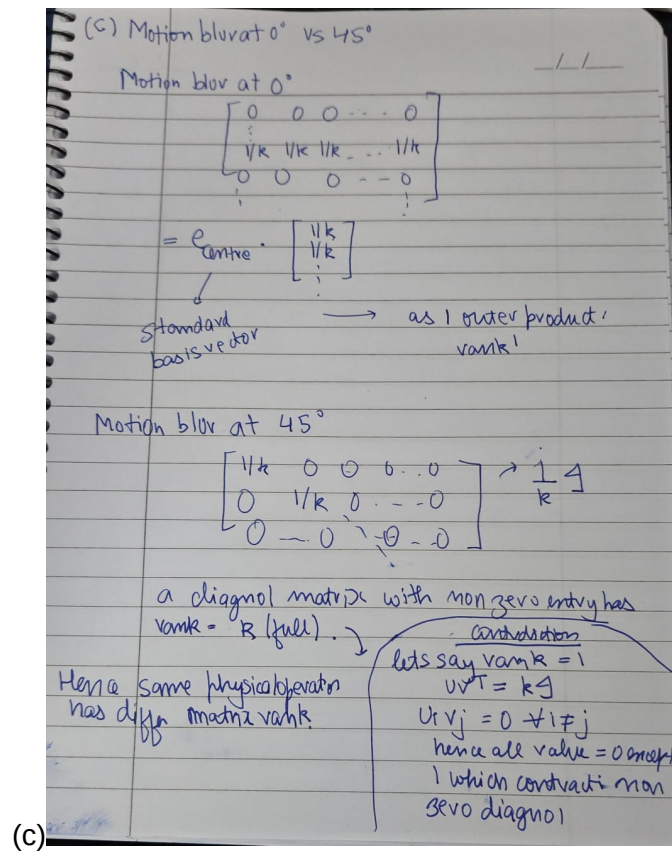
hence $\exists a, b$ such that

$$a \cdot \left(\frac{x^2}{\sigma^2} - 1 \right) e^{-\frac{x^2}{2\sigma^2}} + b \cdot e^{-\frac{x^2}{2\sigma^2}} = 1 \text{ for all } x$$

but Gaussian weighted function $\rightarrow 0$ as $x \rightarrow \infty$
but right side not zero. Hence column
vectors independent. $\rightarrow$ rank = 3

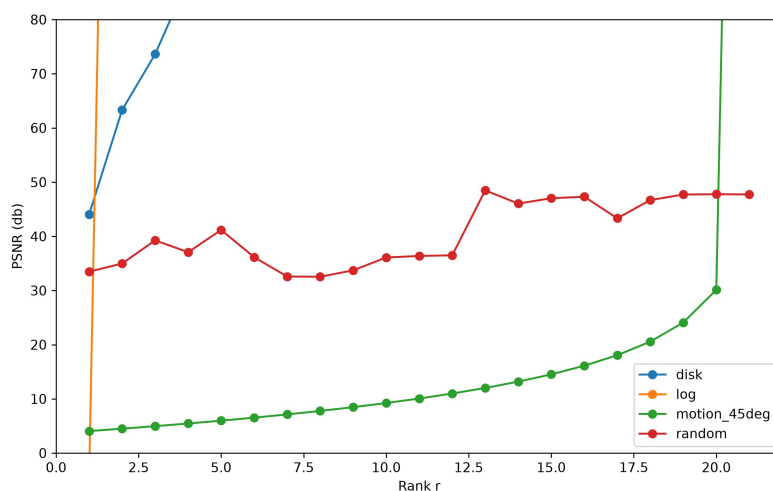
(b)

A constant vector 1 cant be represented as a linear combination of the 2 1D functions appearing in the LoG decomposition hence adding rank-1 term 1.1transpose increasing rank from 2 to 3



A horizontal blur is represented by a single nonzero row, which is separable and rank 1. A 45degree blur places nonzero values along the diagonal, producing a scaled identity matrix whose rows are linearly independent, so its rank is k

1.3



Interpretation of the graph: log jumps to large value (above 80) at r = 2 confirming it has a low rank. Although disk is not rank-1 we can see that its singular values decay rapidly and hence we

reach high PSNR after first few singular components $\sim 3-4$. Motion45 rises very slowly and linearly and only has high PSNR value at $n = 21 = k$.

1.3 (c) Disk and motion45 are both non-separable, but they have some slight differences:

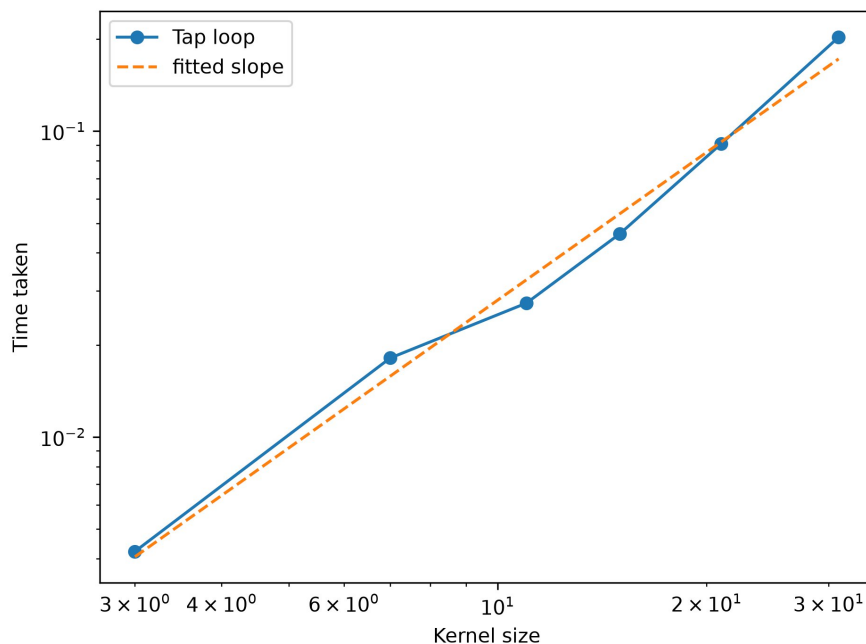
The motion45: is a scaled identity matrix, what this means is that no singular value is more important than the other so dropping even one term loses a significant fraction of the information.

The Disk: while circular disk is not perfectly separable, it is highly symmetric and smooth hence its singular values decay rapidly (there are some more significant than others) hence the first few singular values represent the bulk of the information while others capture the fine details only.

We can predict from the singular values if the spectrum is flat (all values roughly equal) then kernel has no low rank structure and you need all k terms

If the spectrum decays rapidly then the kernel can be easily approximated and small r will show high fidelity.

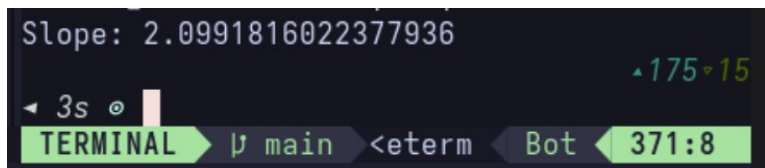
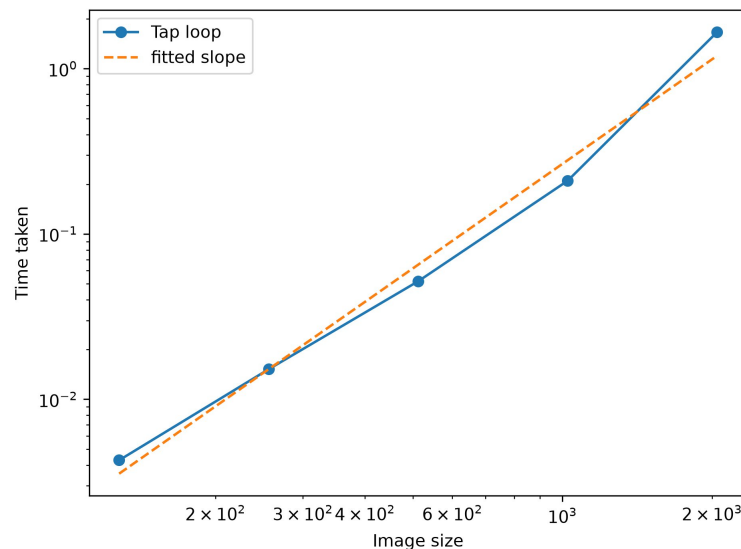
1.4



```
Slope: 1.6056096132881448
◀ 1s ◉
TERMINAL ▶ μ main <eterm ▶ Bot ▶ 311:8
146.15
```

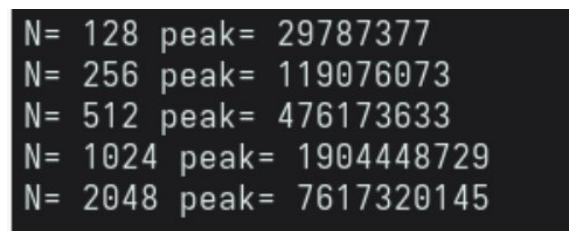

(a)

The idea slope should be 2 however upon conducting the experiment we get the slope to be 1.6. This is because the theoretical scaling of $O(k^2)$ only captures the number of operations, the empirical time however also includes the time taken by python overhead functions like padding function call, memory cache behavior etc.. Cache effects like for small values of k like 3 or 7 kernel can fit in the cache whereas there are extra data access overhead for larger k values. Because of the effect of numpy operations scaling, cpu (if it supports clock speed relative to load) etc the empirical slope doesnt match the theoretically calculated one. So the missing component is the hidden system level operations and optimizations.



For fixed $k = 15$, runtime against image size N produces a fitted log-log slope of 2.099. This follows the theoretical $O(N^2)$ scaling. The slight deviation from 2 is due to memory, cache behavior and other implementation level effects.

Im2col Memory tests



Im2Col has a function where we use the `np.lib.stride_tricks.sliding_window_view` and then create a patch matrix of shape $(x*y, kx*ky)$ hence the amount of memory used is

Size of one float value * (x*y) * (kx*ky) in case of square image and square kernel with float values this is $8 * N^2 * K^2$ bytes.\

Theoretical Logical size with the formula $8 * N^2 * K^2$

N = 128: 28.13 MiB

N = 256, 112.50 MiB

N = 512, 450 MiB

N = 1024, 1.76 GiB

N = 2048, 7.03 GiB

In the case of N = 2048 and K = 15 this value is 7549747200 bytes = 7.03125 GiB. This creates a very high memory limit bottle neck for the im2col method

(b)

Comparison of im2col and tap loop

Testing for n = 512*512 for variable K

```
k= 3 tap : 0.0041 col : 0.0084 speedup: 0.4922
k= 7 tap : 0.0206 col : 0.0237 speedup: 0.8667
k= 11 tap : 0.0458 col : 0.0556 speedup: 0.8227
k= 15 tap : 0.0531 col : 0.088 speedup: 0.6036
k= 21 tap : 0.0979 col : 0.1592 speedup: 0.6146
k= 31 tap : 0.2129 col : 0.2762 speedup: 0.7706
```

4s DIP_Assignment_1 py 3.14.7 19:19

As the value of speedup is less than 1. This means that for my PC for all tested kernel sizes im2col was slower than the tap-loop implementation. The im2col converts the convolution problem to a matrix multiplication which uses the optimized numpy linear algebra. The tap the implementation loops over the kernel taps and operations on the image are performed using the factorized numpy array arithmetic.

One possible explanation for the tap to be faster than the col could be that the single kernel workload associated with im2col outweighs the benefit of the final matrix vector multiplication. This overload nullifies (cancels) the faster matrix multiplication hence making it overall slower.

Conditions where im2col wins: when the patch matrix can be used across multiple filters, so that the cost of the constructing patches becomes lesser than the cost of convolution itself. Currently there is only one kernel and the cost of constructing/accessing the patch representation dominates. These conditions therefore do not hold for the present workload.

(c)

```
N=512: tap=0.0500s separable=0.0091s speedup=5.5066x
N=2048: tap=1.6239s separable=0.3574s speedup=4.5440x
```

7s DIP_Assignment_1 py 3.14.7 19:34

For a rank 1 kernel the 2D convolution takes time HWk^2

where as when it is replaced by two 1D kernels the time taken is $2 * HWk$

Therefor the theoretical speed up is $k/2$

For k = 15 hence we expect the speedup to be ~7.5 times.

As seen from the screenshot even upon doing an empirical analysis we get the separable implementation to be substantially faster in both the $N = 512$ and $N = 2048$. But the speedups are lesser than 7.5 this is because the theoretical speedup only considers the number of arithmetic operations but the empirical values depend on memory access, cache behavior, temporary array size, numpy overhead etc. Also from $N = 512$ to $N = 2048$ both implementation process larger arrays cost of reading/writing intermediate arrays starts to become more important. hence the time taken for memory access starts become more significant leading to a decrease in the speedup. Basically the memory related costs become more significant as N increases and has nothing to do with the separability becoming more / less effective.